

Intuitionist - Horizontal Unified Data Science Code Generation Templates Kit

Peeyush Pashine; Nimshi Venkat; Luis Ek; Vinay Artham

GRT/Data Ventures, MLE Horizontal team

Abstract: As DS (Data Science) based projects and initiatives go through the cycle of Proof of concepts (PoCs), development and deployment. There is huge scope of adopting software engineering practices tuned for data science ecosystem. These principles are applicable for variety of DS use cases and can be defined under the umbrella of Automation, Integration, System design and patterns, Quality, cost optimization and operational excellence. The diversity in various data science use cases tackled by different teams presents a challenge in finding the nuances on mentioned principles, that can be mutually applicable. Our solution tackles it in a pragmatic way and provides a central platform to be leveraged by individual DS teams and projects supporting the entire DS lifecycle. In the below paragraphs the approach mentioned, can generate and setup a data science project in 5 minutes effectively have a targeted savings of 12k - 17.5k USD quarterly for a typical data science product as observed under Data Ventures. In this paper we write in detail about the framework and how we measure the KPIs for its effectiveness.

1. Introduction

Our approach Intuitionist (Horizontal Unified Data science Code Generation template kit) involves addressing distinct domain-specific data science challenges through diverse data science teams, catering to various business use cases. The variability in the data science lifecycle and timelines among these teams stems from specific business requirements.

Consequently, this diversity presents a challenge in standard for organizing code and other artefacts, leading to probable inconsistencies across different projects. The challenges with code structure and organization, scaling up in the later stages of data science life cycle, modularity, parameterization, portability and consistency across various stages and systems. This might increase the tech debts specially when, these projects later bundle up under larger suite of distinct objectives and use-cases. In addition, there is a possibility of missing standardized MLOps[5] practices and established build and deployment strategies. These factors collectively contribute to an environment, where achieving uniformity, code quality and efficiency in delivering impactful solutions becomes challenging and may involve lack of clarity in timeline of shipping the DS products, integrating with other data science, and engineering artifacts due to incompatible interfaces and managing tech debts impacting the effective productivity of the team.

In this paper, we talk in detail about our approach to address the below questions:

a) What if, the information and code artifacts pertaining the above points are available during the starting phase of data science project? b) What if the consistent and effective development, build, and deployment artifacts are auto generated and can be used at the appropriate data science life cycle?

Addressing these points acts crucial to streamline our development practices and enhance overall operational effectiveness.

Our framework *Intuitionist* serves as a valuable tool for generating code artefacts in data science projects. We use cookie-cutter[6] package as a backend along with our custom written hooks, examples, and templates to provide a diverse set of permutations and combination with input parameterization and customization. User can generate code artifacts based on the provided tech stack, project names, submodule names. User can generate the nested structure of data science modules and submodules as desired with build sanity, paths and import statements working as desired. These templates facilitate the automatic generation of test examples, linting tools and related artefacts, contributing to a more streamlined and efficient development process. They play a pivotal role in generating essential build artefacts, supporting continuous integration and continuous deployment (CI/CD) processes. Additionally, these templates extend their functionality to the realm of MLOps, providing the necessary artefacts to enhance the overall management and operationalization of machine learning models. The use of unified templates proves instrumental in promoting consistency, adaptability, and automation across different facets of data science project development.

The framework also incorporates optimization strategies like supporting local development.

2. Background

It is important in a DS project to map the product and business requirements to tech executables. Often these can be categorized on a primary level to batch, real-time or streaming. As mentioned in the previous point, an absence of well-structured code, leveraging standard building and deploying practices generate not only tech debts for the future but also gaps on operational excellence and

opportunities to reduce infrastructure cost. Using *Intuitionist* framework, the generated templates highlight a well-defined structure adhering to standards for project development and deployment, aligning to Walmart's platform strategy and standardization-based tools such as Concord, Looper, Artifactory and GitHub. We also provide integration with Element a powerful Walmart tool used by data scientists.

These templates encompass all relevant code artefacts for various use cases, including:

Batch processing – Generation of an example project comprising of batch data science use-cases for test driven development, building CI/CD and MLOps supported deployment with auto code coverage.

Real-time: Generation of applications supporting real time use-cases for test driven development, building CI/CD and MLOps supported deployment with auto code coverage.

Streaming: Generation of streaming based artifacts for data science use-cases.

Support for DS proof of concepts using WCNPs: Generation of applications with our WCNP templates to quickly deploy data science applications.

In the Impact section of this paper, we talk about how this has resulted in improving certain KPIs related to Data Ventures tech OKRs and overall has helped DS teams to increase productivity and reduce ship time to market.

3. Implementation

In this section framework on generating the code templates[7] based on the user inputs, testing examples, artifacts related to

development, and deployment are explained in detail.

3.1 Template types

Intuitionist supports PoC, development and deployment templates:

- PoC templates (WCNP based)
- Development template
- MLOps Deployment template (batch, real-time and streaming)

As mentioned in Section 1 and 2, framework acts as a tool to generate the project repository based on user's input. We also maintain a defined code structure for the usecase provided. Hence whether it is a WCNP template, development or deployment template, every generated project follows a standard code structure respective to the template used. User can also generate combination of these templates for the desired project.

Data scientists in the initial part of project development perform many experiments and would run it through the product and business teams. In this case agility of solution and hosting it in a place with non-prod with optimum infrastructure usage becomes an important requirement. WCNP templates with how to serve a DS solutions using Streamlit (a popular tool among DS for creating quick user-interfaces) or Django/Flask (serving REST API interfaces) can be used. We provide not only a standard code structure to host services in WCNP machines, but also the code artifacts impacting Key performance indicators (KPIs), mapped to tech OKRs (objectives and key results). Auto generation of *Sonar* project, a framework to detect code quality, coverage and vulnerability mapped to Quality KPI and Engineering excellence (EE) OKR at the pillar xlevel. Support of *Akeyless* for secret/credential storage to enable security KPI and Operational excellence (OE). A *pytest* based testing framework to enable DS write better code

and detect quality issues as the development work continues. To eliminate the uncertainty between individual development setup and WCNP deployment setup, we support and promote our DS teams to use generate Docker artifacts enabling local development and testing. Use of same Docker file during WCNP deployment makes sure of consistency and uniformity in what is being developed by team and what is being deployed, thus mapping it to System reliability KPI and OE OKR. In the coming section, how project specific auto generated CI/CD artifact glues the consistency and uniformity aspect will be addressed.

As an alternative to previous approach, many a times, rather using PoC(s), DS teams need to start developing an MVP product based on translated tech requirements from business/product requirements. In such cases, timeframe for the development is usually longer than PoCs. When the user selects development template to generate a project artifact, options such as type of project, module and submodule names, package building[4] are provided. User parameters as provided appear in places in the generated code artifacts. This helps DS to start the development process and bring in their code without any manual changes required. The import statements in generated artifacts work when the package is built locally or using CI/CD artifact. User inputs become part of CI/CD artifacts as well testing framework based on *pytest* and coverage tool *Sonar*. Like previously mentioned similar to the WCNP templates, Sonar and CI/CD projects are auto created. The entire process of development whether using a local setup using Docker or CI/CD becomes seamless that a DS need not to be focused on leveraging Walmart provided tools for Operational work.

DS development ecosystem can be different from how these artifacts will be deployed at

the end. For an example, DS majorly use Jupyter notebooks for adhoc development and testing. While for production code has to be standardised and follow Object oriented modularity. We can call production ecosystem as MLOps deployment ecosystem in this case and usually is taken up by Machine learning Engineers or Software engineering teams. The aspects of timeline management to translate the DS code to deployment pattern is a usual necessity and impacts DS solution shipment roadmap. Our MLOps deployment template comes handy and is decoupled from DS development workflows from day 1. This enables Machine learning engineers to start working in parallel to DS development if the expectation of the solution going to deployment one day grows based on stakeholder interactions. Along with development project generation, an option to generate MLOps code artifacts is provided to the user at the same time. However, MLOps code artifacts can also be generated later at any point in time.

MLOps templates further can be categorized into Batch, real-time and streaming patterns. We leverage Walmart's element platform underneath to support MLOps deployment. The generated MLOps artifacts can be present in the same repository where development artifacts were generated, or it another repo for MLOps work can be used based on the team's discretion. We support multiple MLOps pipelines (e.g. training and inference) in the same repository. Similar to other two templates mentioned before, user has an option to customize desired generated MLOps artifacts based on Project type, use-case, option to build custom containers.

3.2 Artifacts generation, Development and Deployment flow

We use hooks supported by *cookiecutter* engine to write pre code gen and post code gen customization according to user

provided inputs, which involves, adding files and directories to the path, removing, make utilities, customizing CI/CD artifacts etc. The generation mechanism is explained in the Fig2.

To support development and deployment workflows for DS teams, Trunk[3] based CI/CD with code versioning using Github and version control builds are supported. The user customized auto generated CI/CD artifact (*looper*, a Walmart offering) leverage the generated Dockerfile for both development and MLOps deployment. In this manner consistency between what is being developed in individual machines and what runs in production is maintained. DS teams use feature and development branches, while for production main and release branches are used. The Github branching strategy[1] hence is tightly coupled with CI/CD instructions in *looper*. Figure 3 depicts how generated code artifacts plays out from the lens of DS development and deployment workflow-wise.

Intuitionist - Horizontal Unified Data Science Code Generation Templates Kit

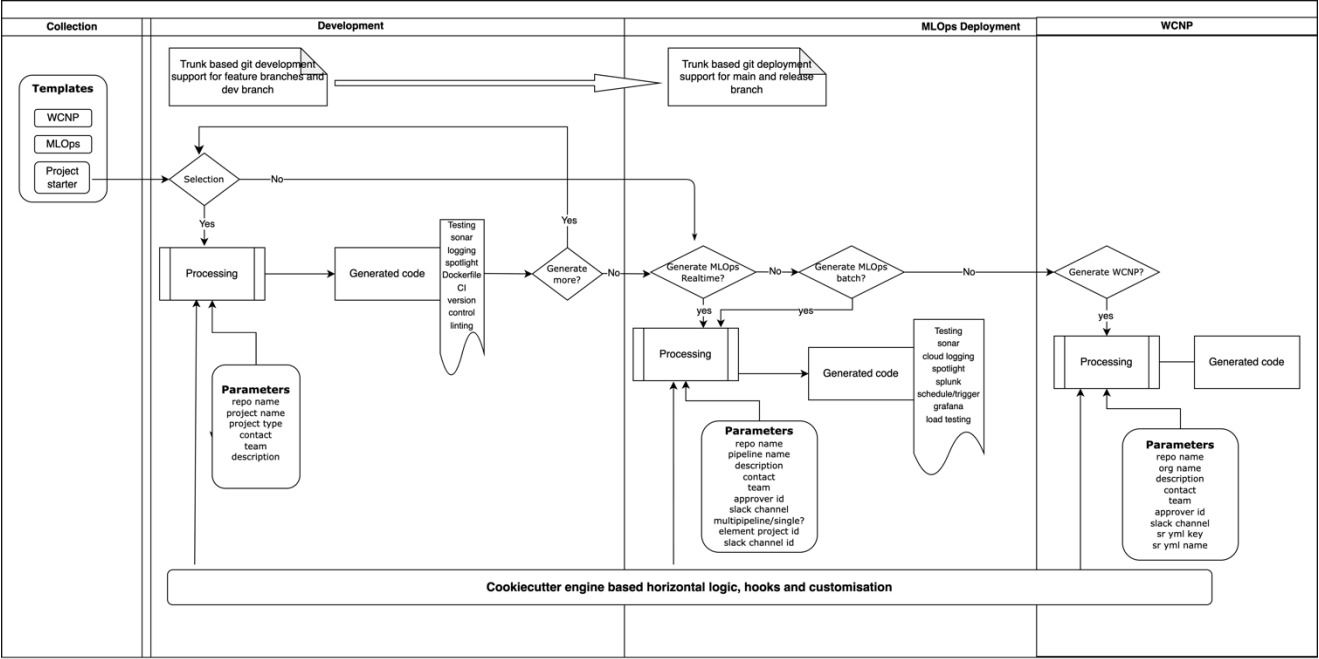


Figure 1. Artifacts configuration and generation flow

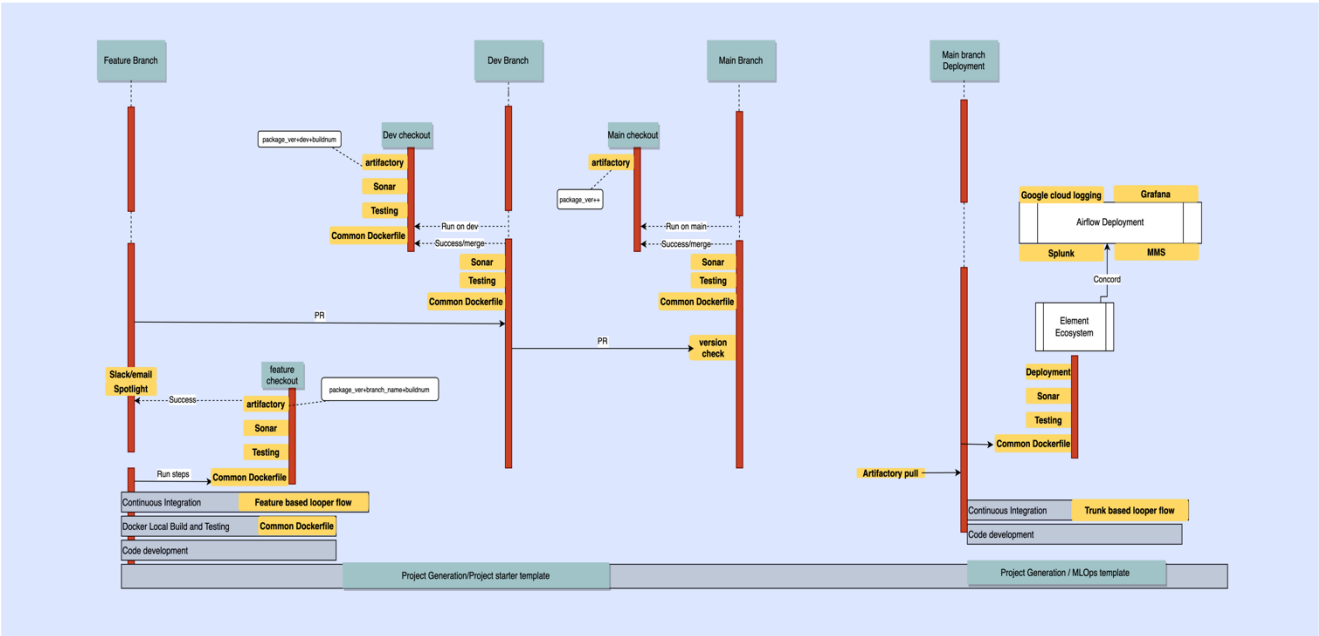


Figure 2. Development and Deployment flow with generated artifacts

3.3 Local setup for development

The generated project also has Docker artifact enabling local setup for development, so that data science can leverage local machines without spinning up the clusters for POCs and development [1]. This helps both in cost reduction as well as speeding up the development process [2]. Fig. 3 depicts the flow of development to production both from local and remote machines aspect.

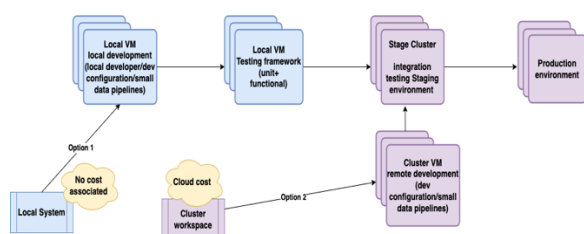


Figure 3. Generated Docker artifact for local development

3.2 Notification artifacts

In case of development and deployments, our framework provides integration to tools such as xMatters, Spotlight for notification on alerts level set from the code artifact. This maps to OE OKR for the project.

4. Impact

In several projects leveraged by DS teams in Data Ventures such as Channel performance, Shelf intelligence, Digital landscape, Customer voice and Customer support the impact has been seen with quick deployment in coherence with local development and cloud development-based artifacts making the entire process more streamlined.

This has resulted significant 50% increase in the adoption of code quality standards from a baseline of 10%, driven by robust testing procedures and comprehensive code coverage. Simultaneously, there has

been a noteworthy 50% uptake in leveraging auto-build capabilities from a baseline of 5%, providing crucial support throughout the development cycle. This helps the teams to target zero-defect code for shipment, an early detection of issues, reducing the tech-debt, better integration with engineering and other products, reduce RCAs, better budget planning to support scale. In the Figure 4, for different product areas, KPIs are captured.

Projects in Area	Code Metrics	Build metrics	Quality metrics
Customer voice	- PR Cycle reduction by 300% - PR Review time reduction by 250%	- Build success rate up by 40% - PR build success rate up by 57%	- Code coverage up by 30% - Security vulnerability reduction by 20%
Channel Performance	- PR reviews in progress reduction by 500% - PR Cycle 7 hrs, in recommended range < 24 hrs - PR Review time 9 hrs, in recommended range < 12 hrs - PR reviews in progress 1.83, recommended range < 47.5	- Average build duration down by 50% - Build Success Rate 93.75% - PR Build Success Rate improving by 30%	- Production Defect Arrival Rate 0.48 /week - Security vulnerability reduction by 20%
Digital landscape	- PR Cycle reduction by 400% - PR Review time reduction by 350% - PR reviews in progress reduction by 600%	- Build Success Rate 93.75% - PR Build Success Rate improving by 30%	- Code coverage up by 10% - Security vulnerability reduction by 20%

Figure 4. Impact KPIs for different DV DS areas

In the realm of MLOps deployment, there has been a commendable 20% adoption rate thus far from the baseline of 0%, indicating a promising integration of these practices. This helps to automate the DS pipeline as per novel practices, better monitoring, governance and in turn address Walmart's goal of operational excellence. Additionally, there is a 30% adoption rate from baseline of 0%, for utilising local development environments based on horizontal Docker images - A strategy seamlessly applicable to both local development and the CI/CD pipeline, that results in drastic reduction of cost and uniformity among various stages of data science lifecycle. These strategies collectively helping data science product in data ventures, operating at scale to save up to 17500 USD on a quarterly basis.

5. Conclusion

Our framework *Intuitionist* has played a vital role cutting across different DS verticals in Data Venture to help DS teams improve productivity, operational excellence and quality. This in turn has reduced the pain points data scientist and machine learning engineer may face during the various stages of DS lifecycle and ship the products faster. Our framework can be applied to other DS teams outside Data Ventures, and act under the umbrella of AI phantomization and artifacts tools.

6. Bibliography

- [1 gitkraken, "Github branching strategy,"
] [Online]. Available:
<https://www.gitkraken.com/learn/git/best-practices/git-branch-strategy>.
- [2 Jenkins, "Multibranch," [Online].
] Available:
<https://www.jenkins.io/doc/book/pipeline/multibranch/>.
- [3 Atlassian, "Trunk based development,"
] [Online]. Available:
<https://www.atlassian.com/continuous-delivery/continuous-integration/trunk-based-development>.
- [4 P. org, "packaging python projects,"
] [Online]. Available:
<https://packaging.python.org/en/latest/tutorials/packaging-projects/>.
- [5 Wikipedia, "MLOps," [Online].
] Available:
<https://en.wikipedia.org/wiki/MLOps>.
- [6 A. R. Greenfeld, "Cookiecutter package,"
] [Online]. Available:
<https://github.com/cookiecutter/cookiecutter>.

- [7 c. docs, "Better project templates,"
] [Online]. Available:
<https://cookiecutter.readthedocs.io/en/stable/>.
- [1 gitkraken, "Github branching strategy,"
] [Online]. Available:
<https://www.gitkraken.com/learn/git/best-practices/git-branch-strategy>.
- [2 Jenkins, "Multibranch," [Online].
] Available:
<https://www.jenkins.io/doc/book/pipeline/multibranch/>.
- [3 Atlassian, "Trunk based development,"
] [Online]. Available:
<https://www.atlassian.com/continuous-delivery/continuous-integration/trunk-based-development>.
- [4 P. org, "packaging python projects,"
] [Online]. Available:
<https://packaging.python.org/en/latest/tutorials/packaging-projects/>.
- [5 Wikipedia, "MLOps," [Online].
] Available:
<https://en.wikipedia.org/wiki/MLOps>.
- [6 A. R. Greenfeld, "Cookiecutter package,"
] [Online]. Available:
<https://github.com/cookiecutter/cookiecutter>.
- [7 c. docs, "Better project templates,"
] [Online]. Available:
<https://cookiecutter.readthedocs.io/en/stable/>.